Blood Bank System — 1-Day Development Blueprint

Since you and your partner will be building everything from scratch tomorrow, here is the complete blueprint. It divides the work perfectly into two halves so you can both code in parallel.

Overview

Stack to use: Node.js, Express, EJS, MySQL. **Initial Step:** Build the database schema first, insert dummy data, test your triggers/procedures, then build the web app on top.

👤 Person 1: Donors & Inventory

Your job is the core donor management and the physical blood stock.

1. Database Tasks (Morning)

You need to write SQL to create the following:

Tables:

- `blood_group` (id, blood type name like 'A+', 'O-')
- `donor` (id, name, group_id, phone, last_donation_date, is_eligible)
- `blood_inventory` (id, group_id, units_available, expiry_date)

Advanced SQL:

- **View** `vw_blood_stock` : Write a query that groups by blood type and sums the available units.
- **View** `vw_donor_history` : Write a query that joins donors with donations to show their history.
- **Stored Procedure** `sp_check_eligibility` : Logic to check if 90 days have passed since a donor's last donation date.
- **Trigger** `before_donation_insert` : Throw an error (using `SIGNAL SQLSTATE`) if an ineligible donor tries to donate.
- **Trigger** `after_donation_insert` : Automatically increase units in `blood_inventory` after a successful donation.

2. UI & Backend Tasks (Afternoon)

Set up the Node/Express server and build these pages:

- **Login Page**: Simple username/password form for admins to authenticate.
- **Dashboard**: Show total metrics (Total Donors, pending requests) and a snapshot of the `vw_blood_stock` .
- **Donors Page**: A list of all donors. Display a badge showing if they are eligible or cooling down.
- **Add Donor Page**: Form to insert a new donor into your `donor` table.
- **Inventory Page**: A colour-coded table showing blood stock (e.g., Red for low stock (< 5 units), Green for good stock).

👤 Person 2: Donations, Hospitals & Requests

Your job is managing the inflow (donations from donors) and outflow (requests from hospitals).

1. Database Tasks (Morning)

You need to write SQL to create the following:

Tables:

- `admin_user` (id, username, password_hash, role)
- `hospital` (id, name, address, contact_person)
- `donation` (id, donor_id, units_donated, donation_date, status)
- `blood_request` (id, hospital_id, urgency, units_required, status)
- `request_fulfillment` (id, request_id, inventory_id, fulfilled_by_admin_id)

Advanced SQL:

- **View** `vw_pending_requests` : Show only requests where the status is 'Pending', ordering them by Urgency (Critical first).
- **Stored Procedure** `sp_fulfill_request` : Wrap this in a `TRANSACTION` . It should update the request status to 'Fulfilled' AND insert a new row into `request_fulfillment` . If one fails, `ROLLBACK` .
- **Trigger** `after_fulfillment_insert` : Automatically deduct units from `blood_inventory` when a request is fulfilled.

2. UI & Backend Tasks (Afternoon)

Build these pages connecting to your tables:

- **Hospitals Page**: View existing hospitals and a form to add new ones.
- **Donations Page**: Form to record a new donation. When you insert into this table, Person 1's `after_donation_insert` trigger will fire.
- **Requests Page**: View pending requests from hospitals (using your view). Add an "Approve" button that calls `sp_fulfill_request` .
- **Reports Page**: Create a simple bar chart (you can use Chart.js via CDN) showing the blood stock distribution.

🚀 Game Plan for Tomorrow

Morning: Database Design

1. Both log into MySQL phpMyAdmin separately.
2. Person 1 creates their tables, Person 2 creates theirs.
3. Make sure the foreign keys match (e.g., Person 2's `donation` table needs Person 1's `donor_id`).
4. Write your views, procedures, and triggers. Insert some dummy data to test them.

Afternoon: Backend & UI

1. Person 1 generates the base Express server (`npm init` , `npm install express ejs mysql2`) and sets up a shared layout (`views/layout.ejs`).
2. Person 1 builds route handlers (e.g., `routes/donors.js`) and EJS views for donors.
3. Person 2 builds route handlers (e.g., `routes/requests.js`) and EJS views for requests.

Evening: Integration & Demo Prep

1. Merge your code together.
2. Test the triggers from the app: manually add a donation and verify the inventory number goes up.
3. Finalize the demo script.

🎯 Demo Order (What to show the evaluator)

1. Open phpMyAdmin → show all 8 tables.
2. Person 1 runs `vw_blood_stock` query → explains normalization.

3. Person 2 runs `vw_pending_requests` query → explains ordering by urgency.
4. Person 1 calls `sp_record_donation` missing parameters or ineligible donor → show trigger throwing an error.
5. Person 2 calls `sp_fulfill_request` → show trigger deducted inventory correctly.
6. Open browser → Walk through the pages you built (login → dashboard → donors → requests) and approve a request on screen.